WORKING WITH INKHAVEN

Publish Your Book to the Web

A Plain-Language Guide to Inkhaven's HTML Export — Styling, Templates, and Going Live

Vladimir Ulogov

2026 · examples assume Inkhaven 3.0.0 or newer

Contents

CHAPTER 0

0

Before we begin

You have written a book in Inkhaven. Now you want other people to read it — not as a file they must download and open in a particular program, but as something they can simply *visit*, the way they visit any page on the internet. That is what this short book teaches: how to turn your manuscript into a website, how to make that website look the way you want, and how to put it somewhere the world can reach it.

You do not need to be a web designer, and you do not need to be an Inkhaven expert. Everything here is explained from the ground up. Where a word has a precise meaning — and the web is full of such words — it is defined the first time it appears, in a box like this one:

TERM **Website**

A collection of pages, written in a language called HTML, that a web browser (Safari, Chrome, Firefox) can display. A website can live on the internet for anyone to visit, or sit in a folder on your own computer. Inkhaven builds the second kind, and you decide whether to put it online.

TERM **HTML**

HyperText Markup Language — the format web pages are written in. You will never have to write HTML by hand: Inkhaven writes it for you, from the same manuscript you already have. HTML is just text, so a website is really just a folder of text files and pictures.

What you will end up with

One command turns your book into a *folder*. Inside that folder is a small website: one page for each chapter, a table of contents down the side, a clean reading design, and copies of any pictures your book uses. You can open it on your own machine to check it, and when you are happy, you copy that folder to a web host and it is live.

Crucially, the site Inkhaven produces is *self-contained*.

TERM **Self-contained**

Everything the website needs to display is inside its own folder — the design, the pictures, the text. It does not reach out to anyone else's computer to load a font or a style. That means it works with no internet connection at all, it will still work years from now, and nothing outside it can change how it looks. You can email the folder to someone and it just works.

How this book is arranged

We begin with the single command that makes a site, and look at what it produces. Then we cover the terminal commands in full, so you know every option you have.

The middle of the book is about making the site *yours*: its colours and type (styling), its layout (templates), and the little pieces of text that appear throughout (variables). Near the end we talk about choosing exactly what goes on the site, and finally how to put it on the internet.

You can read straight through, or jump to the chapter you need. Each one ends with a short recap of what it covered.

INSIGHT

The whole feature rests on one idea: you already wrote the book. Publishing it to the web should not mean writing it again in a new form. So Inkhaven reads the very same manuscript you have been editing and renders it as a website — and every time you change the book and export again, the website simply catches up. There is no second copy to keep in sync.

WHAT YOU LEARNED

- A **website** is a folder of HTML pages a browser can display; you decide whether to put it online.
- Inkhaven builds the site from your existing manuscript — no second copy, no HTML by hand.
- The site is **self-contained**: it needs nothing from the outside to display, works offline, and travels as a single folder.
- You need no web or Inkhaven expertise; every term is defined as it appears.

PART I

Your First Website

CHAPTER 1

1

Your first website

Let us make a website right now, before we explain anything else. You will see how little it takes, and the rest of the book will make sense against that first result.

The terminal

Inkhaven's export runs from the *terminal*.

TERM **Terminal**

A window where you type commands to your computer as text, instead of clicking buttons. On macOS it is the app called *Terminal*; on Windows, *PowerShell* or *Windows Terminal*; on Linux, your *console*. You type a line, press Return, and the computer does it.

If you have used Inkhaven from the terminal before — to start it, or to export a PDF — this is the same window. Open it, and move into the folder that holds your project (the folder with your book in it). Then type one line:

AT THE TERMINAL

```
inkhaven export html --output site
```

Press Return. In a moment Inkhaven prints something like `wrote HTML site to site`, and you are done. You have a website.

TERM **`inkhaven`**

The program itself — the same one that runs the editor. Typed at the start of a terminal command, it means "Inkhaven, do the following." Everything after it tells Inkhaven *what* to do; here, `export html`.

What just happened

The word `--output site` told Inkhaven where to put the website: in a new folder called `site`, next to your project. Look inside it and you will find something like this:

AT THE TERMINAL

```
site/
  index.html          the front page
  ch01-beginnings.html  one page per chapter
  ch02-the-journey.html
  theme.css           the design (colours, type, layout)
  assets/             copies of your pictures
```

Every one of those is an ordinary file. The `.html` files are your pages; `theme.css` holds the look; `assets` holds your images. That whole folder *is* the website.

TERM `--output` (or `-o`)

The part of the command that names the destination folder. `--output site` and the short form `-o site` mean the same thing. If the folder does not exist, Inkhaven creates it; if it does, Inkhaven writes the fresh site into it.

Looking at it

To see your site, open the file `index.html` in a web browser — double-click it, or drag it onto your browser window. It opens like any web page: your book's title, a list of chapters down the side, and your first chapter ready to read. Click a chapter in the sidebar to jump to it; use the arrows at the foot of each page to move to the next.

You are looking at a real website, sitting on your own computer. Nobody else can see it yet — that is the final chapter — but everything is here.

TRY IT

Export your book with the command above, open `site/index.html`, and click through a chapter or two. Notice that the pictures are there, the chapters are in order, and the whole thing reads cleanly. That is the default design, and it is a perfectly good place to stop — you can publish exactly this. The rest of the book is about the times you want more.

NOTE

Your site comes with **search** built in — a box at the top of the sidebar that finds any word across every page as you type, working entirely on the reader's own machine with no server. It is on by default; if you ever want it off, set `docs: { html: { search: false } }` in your settings.

PITFALL

If the terminal answers with `command not found: inkhaven`, the program is not on your system's list of commands yet. That is an installation matter, not an export one — the same thing that lets you start the editor by typing `inkhaven`. Once the editor starts from the terminal, export will too.

WHAT YOU LEARNED

- `inkhaven export html --output site` turns your book into a website in the `site` folder.
- The folder holds one `.html` page per chapter, a front page, the `theme.css` design, and an `assets` folder of pictures.
- Open `index.html` in any browser to read the site on your own computer.
- The default design is publishable as-is; everything after this is optional refinement.

PART II

The Commands

CHAPTER 2

2

The command in full

You have met the short form. Now let us lay out the whole command, every option it takes, and when you would reach for each. None of these is required for a good site — the bare `inkhaven export html -o site` is complete — but each one answers a real question you may eventually have.

The general shape is always the same:

> **AT THE TERMINAL**
>
> ```
> inkhaven export html [options]
> ```

TERM **Option (or flag)**

An extra word on a command, beginning with two dashes, that adjusts what it does — like `--output`. Some options take a value after them (`--output site`); some, called *switches*, stand alone. The order of options does not matter.

Where the site goes — `--output`

Covered already, and the one option you will always use: `--output <folder>` (short `-o <folder>`) names the destination. A fresh export overwrites the pages in that folder, so pointing two exports at the same folder is normal — the second simply replaces the first.

AT THE TERMINAL

```
inkhaven export html -o public
```

Which book — `--book-name`

A project can hold more than one book. If yours holds only one, Inkhaven exports it without being told. If it holds several, name the one you mean:

AT THE TERMINAL

```
inkhaven export html -o site --book-name "The Drowned Atlas"
```

NOTE

The name is the book's title as it appears in your project's tree, in quotation marks if it contains spaces. If you have one book, you never need this.

Which edition — `--profile`

This is the most powerful option, and it has a chapter's worth of ideas behind it, so here we only introduce it. A single manuscript can carry more than one *edition* — a short version and a full one, a beginner's path and an expert's. You mark paragraphs for an edition inside the editor, and then at export time you ask for that edition:

AT THE TERMINAL

```
inkhaven export html -o site --profile edition=full
```

You can ask for more than one dimension at once:

AT THE TERMINAL

```
inkhaven export html -o site --profile edition=full --profile
audience=expert
```

Chapter 6 explains how to mark the paragraphs and how the matching works. For now, know that `--profile` is how you publish *one slice* of a manuscript that contains several.

Custom design — `--templates` and `--eject-templates`

By default the site is built from a set of design files bundled inside Inkhaven — you never see them, and you do not need to. When you want to change the design beyond what the settings allow, you work with your own copy of those files. Two options make that possible.

The first *writes the bundled files out* so you have something to edit:

AT THE TERMINAL

```
inkhaven export html --eject-templates my-design
```

TERM **`--eject-templates <folder>`**

Writes Inkhaven's built-in design files into the folder you name, then stops without exporting. This is how you get a starting point to customise — especially important if you installed Inkhaven as a finished program, because then the files exist only inside it until you ask for them. "Eject" because you are ejecting the built-in copies out to where you can reach them.

The second *uses* a folder of design files when it exports:

AT THE TERMINAL

```
inkhaven export html -o site --templates my-design
```

TERM **`--templates <folder>`**

Tells Inkhaven to take the design from the folder you name instead of its built-in defaults. Any file you did not change still falls back to the default, so you only keep the files you actually edited.

The natural rhythm is: eject once, edit the few files you care about, then export with `--templates` pointing at them. Chapters 3 and 4 are entirely about what is inside those files.

The settings file, instead of options

Everything you can pass as an option, you can also set once in your project's settings file, so you do not retype it every export.

TERM `inkhaven.hjson`

Your project's settings file, sitting in the project folder. It records your preferences in a friendly, readable format (the next chapters lean on it heavily). Options you pass on the command line always win over what is written here, so the file sets your usual habit and an option overrides it for one run.

For example, to make `my-design` your permanent template folder without typing `--templates` every time:

EDIT · inkhaven.hjson

```
docs: {
  html: {
    template_dir: "my-design"
  }
}
```

With that saved, a plain `inkhaven export html -o site` uses your custom design. We will add more settings under `docs: { html: { … } }` as the book goes on.

WHAT YOU LEARNED

- `--output <folder>` (or `-o`) names where the site is written; you always use it.
- `--book-name` picks a book when a project holds several.
- `--profile dimension=value` publishes one edition of a manuscript that carries many (Chapter 6).
- `--eject-templates <folder>` writes the design files out to edit; `--templates <folder>` exports using them.
- Anything you can pass as an option can live permanently under `docs: { html: … }` in `inkhaven.hjson`.

PART III

Making It Yours

CHAPTER 3

3

Colours, type, and the look

The default site is deliberately plain and warm — cream paper, earth-brown headings, a serif face for reading. When you want it to look like *yours*, you change one file. This chapter is about that file and the small language it is written in.

What styling is

The words of your book and the way those words *look* are two separate things. The same chapter can be set in any typeface, any colour, any width. That separation has a name on the web.

TERM **CSS**

Cascading Style Sheets — the language that describes how a web page looks: its colours, its fonts, its spacing, its layout. HTML says *what* the page contains; CSS says how it should *appear*. They are kept in separate files so you can restyle a site completely without touching a word of its content.

In your exported site, all of the look lives in one file: `theme.css`. Change it, and the whole site changes with it.

Getting your own copy

You do not edit the site's `theme.css` directly — a fresh export would overwrite it. Instead you edit your *source* copy, the one Inkhaven builds from. Write the defaults out first, as we saw in Chapter 2:

AT THE TERMINAL

```
inkhaven export html --eject-templates my-design
```

Inside `my-design` you will find two folders — `functional` and `theme` — and the file you want is `my-design/theme/theme.css`.

TERM **The `functional` and `theme` split**

Inkhaven keeps its design files in two piles. `theme` is the *look*: the stylesheet and small visual pieces like the header and footer. `functional` is the *machinery*: the page skeleton and the navigation logic. You restyle by touching `theme` and leaving `functional` alone — so you can change every colour on the site without any risk of breaking how the chapters link together.

The colours, in one place

Open `theme/theme.css`. Near the top you will see a block that begins `:root {` and a list of names that start with two dashes:

EDIT · theme/theme.css

```
:root {
  --bg:      #fbf6ea;   /* page background — warm cream */
  --text:    #1e1a15;   /* body text — near-black ink */
  --accent:  #8a5a2b;   /* headings and links — sienna */
  --code-bg: #f2ecdd;   /* the tint behind code */
}
```

TERM **A CSS variable**

A named colour or value, written `--name`, defined once and used everywhere the design refers to it. Change `--accent` in this one place and every heading and link on the site changes with it. The `#8a5a2b` is a colour written as a *hex code* — a `#` followed by six characters standing for red, green, and blue.

Suppose you want a deep teal instead of the sienna accent. Change the one line:

EDIT · theme/theme.css

```
  --accent: #2f6668;   /* was #8a5a2b */
```

Export with your design, and every heading and link across the whole site is now teal. You changed one value; the "cascade" in the name did the rest.

AT THE TERMINAL

```
inkhaven export html -o site --templates my-design
```

NOTE

The default theme already answers to light and dark screens: readers whose devices are set to dark mode get a dark version automatically. Those dark colours live a little further down the same file, in a block marked `prefers-color-scheme: dark`. Adjust them the same way — by changing the values.

Type

Just below the colours you will find the fonts:

EDIT · theme/theme.css

```
  --serif: "Iowan Old Style", Palatino, Georgia, serif;
  --mono:  "SFMono-Regular", Menlo, monospace;
```

TERM **A font stack**

A list of typefaces, best first. The reader's browser uses the first one their computer actually has, and falls back to the next. Ending with a general word like `serif` guarantees *something* appropriate always shows. Listing common, already-installed fonts is what keeps the site self-contained — it never has to fetch a font from elsewhere.

To set the reading face to Georgia everywhere, put it first:

EDIT · theme/theme.css

```
  --serif: Georgia, "Times New Roman", serif;
```

PITFALL

It is tempting to reach for a beautiful font hosted on the web and link to it. Resist it. The moment a page loads a font from someone else's server, your site is no longer self-contained — it breaks offline, and it depends on that server staying up forever. Style with fonts readers already have, and the site stays whole. If you truly need a special face, the safe path is to bundle the font file into your design folder yourself, not to link out.

INSIGHT

Almost all the styling you will ever want is a matter of changing values that are already named for you at the top of `theme.css` — the background, the accent, the fonts. You rarely need to understand the rest of the file. Find the name, change the value, export, look. That loop is the whole craft of restyling.

WHAT YOU LEARNED

- **CSS** describes how a page looks; all of your site's look lives in `theme.css`.
- Edit your ejected **source** copy under `theme/`, not the site's output copy.
- Colours and fonts are named **CSS variables** at the top of the file — change the value in one place, the whole site follows.
- Keep the site self-contained: style with fonts readers already have, or bundle a font file; never link to one on the web.

CHAPTER 4

4

The page, and how it is built

Colours and fonts change how the site looks. *Templates* change how each page is put together — what goes at the top, what sits in the sidebar, what appears at the foot of every chapter. This is a step deeper than styling, and you may never need it. But when you want a logo in the corner or a line in every footer, this is where it lives.

What a template is

Every page on your site has the same frame: the same header, the same sidebar, the same footer, wrapped around *different* chapter text. Inkhaven does not store that

frame separately for each page. It stores it once, with a blank where the chapter goes, and fills the blank in for each chapter as it builds the site. That reusable frame is a template.

> TERM **Template**
>
> A page with blanks in it. Inkhaven keeps one template for the whole site and, for each chapter, fills the blanks with that chapter's title, text, and navigation. Write the frame once; get a consistent page every time.

The main template is `functional/page.html` in your ejected design folder. It is mostly ordinary HTML — the same kind of text your finished pages are made of — with blanks marked in a small language called Jinja.

> TERM **Jinja**
>
> A simple language for writing the blanks in a template. Two marks are almost all of it: `{{ something }}` means "put the value of *something* here," and `{% … %}` wraps an instruction like "repeat this for every chapter." Everything outside those marks is plain HTML that passes through unchanged.

What you can put in the blanks

When Inkhaven fills a template, it hands it a set of named values to draw from. You refer to them inside `{{ }}`. The ones you will use most:

`book.title`
Your book's title.

`page.title`
The title of the chapter being built.

`page.content`
The chapter's text, already turned into HTML. This is the big blank.

`nav`
The list of chapters, for building the sidebar.

`site`
Your own values from a settings file — title, author, anything you like. This is Chapter 5.

`labels`
The small interface words (“Contents”, “Search”), already translated to your book’s language.

So a line in the template like this —

EDIT · functional/page.html

```
<title>{{ page.title }} · {{ book.title }}</title>
```

— becomes, on the chapter page for “Beginnings” of a book called “The Drowned Atlas”:

EDIT · the built page

```
<title>Beginnings · The Drowned Atlas</title>
```

Inkhaven simply replaced each blank with its value.

A repeat, for the sidebar

The sidebar lists *every* chapter, but you do not write each one — you write the pattern once and Jinja repeats it. In `page.html` you will find something close to this:

EDIT · functional/page.html

```
<ul>
  {% for item in nav %}
  <li><a href="{{ item.href }}">{{ item.title }}</a></li>
  {% endfor %}
</ul>
```

> **TERM A `for` loop**
>
> The instruction `{% for item in nav %} … {% endfor %}` means "for each chapter in the list, produce this once." Inside, `item.title` and `item.href` are that chapter's title and its page's address. One pattern, however many chapters you have.

You rarely need to touch this — it already builds a correct sidebar. It is shown so that the file is not a mystery when you open it.

An edit you might actually make

The footer at the bottom of every page is the friendliest thing to change, and it lives in `theme/footer.html` — a tiny template of its own. To add a line of your own to every page's foot, open it and add plain HTML:

> **EDIT · theme/footer.html**

```
<footer class="site-footer">
  <p class="footer-built">{{ labels.built_with }} Inkhaven</p>
  <p>First edition · Printed nowhere · Read everywhere.</p>
</footer>
```

Export with your design, and that line now sits at the foot of every chapter. You wrote it once; the template put it on every page.

> **NOTE**
>
> `theme/header.html` is the matching piece at the top of the sidebar — the book's title and subtitle. Edit it the same way. Because both are in `theme`, they count as *look*, not machinery: safe to change freely.

PITFALL

If you delete one of the `{{ … }}` blanks the site depends on — the big `{{ page.content }}` one especially — your pages will come out missing their text. When you edit a template, *add* around the blanks; do not remove the ones that were there. If a page comes out wrong, eject a fresh copy and compare.

WHAT YOU LEARNED

- A **template** is the page frame, written once with blanks that Inkhaven fills for each chapter.
- **Jinja** marks the blanks: `{{ value }}` inserts a value, `{% for … %}` repeats a pattern.
- Useful values include `book.title`, `page.title`, `page.content`, `nav`, `site`, and `labels`.
- The footer (`theme/footer.html`) and header (`theme/header.html`) are the easiest templates to make your own.

CHAPTER 5

5

Your own values, in one place

Your name. The book's subtitle. A copyright line. A tagline. These small pieces of text want to appear on the site, often in more than one spot, and you want to write each of them *once*. That is what variables are for. There are two kinds in Inkhaven — one for the site's frame, one for the book's own text — and this chapter covers both.

A file of your values

You keep your site-wide values in a small file in your project, written in a format built for exactly this.

TERM **HJSON**

A gentle version of a common data format called JSON, made comfortable to write by hand. You list `name: value` pairs; you may leave out most quotation marks; you may add `#` comments; and trailing commas are forgiven. It is the same format `inkhaven.hjson` uses. Think of it as a tidy list of labelled values.

Create a file called `html.hjson` in your project folder and put your values in it:

EDIT · html.hjson

```
{
  title:    "The Drowned Atlas"
  subtitle: "A field guide to a sunken world"
  author:   "Mara Vendt"
  footer:   "© 2026 Mara Vendt"
}
```

Each line names a value. Nothing here is required and the names are your choice — these four are simply common ones.

How the values reach the page

Here is the part worth understanding slowly. Inkhaven reads `html.hjson`, and hands everything in it to your templates under the single name `site`. So a value written `author: "Mara Vendt"` in the file becomes `site.author` in a template.

TERM **Translating HJSON to Jinja**

The bridge between the two formats. A pair you write in `html.hjson` as `author: "Mara Vendt"` arrives in your templates as the Jinja value `site.author`. The file supplies the values; the templates spend them. `site` is simply the basket everything in the file lands in.

So to show your name in the sidebar header, open `theme/header.html` and refer to it:

EDIT · theme/header.html

```
<div class="brand">
  <span class="brand-title">{{ site.title }}</span>
  <span class="brand-sub">{{ site.subtitle }}</span>
</div>
```

When Inkhaven builds the site, `{{ site.title }}` becomes `The Drowned Atlas` and `{{ site.subtitle }}` becomes `A field guide to a sunken world` — the values you wrote once in `html.hjson`. Change the subtitle in the file, export again, and it changes everywhere the template used it.

NOTE

The bundled templates already reach for `site.title`, `site.subtitle`, `site.author`, and `site.footer` in sensible places, so simply *creating* `html.hjson` with those names fills them in — even before you touch a template. Add your own names (`site.tagline`, `site.edition`, anything) and use them wherever you like.

TRY IT

Make an `html.hjson` with a `title` and a `subtitle`, then export with a plain `inkhaven export html -o site` — no template editing at all. Open the site and see your title and subtitle in the sidebar. You have just fed values from a data file into a web page.

The other kind: values inside your book

The variables above live in the site's *frame*. There is a second kind that lives in the *text of the book itself* — for a value you repeat in your prose and may need to change everywhere at once, like a product's name or a version number.

You define these under `docs: { variables: … }` in `inkhaven.hjson`:

EDIT · inkhaven.hjson

```
docs: {
  variables: {
    product: "Tidewalker"
    version: "3.0"
  }
}
```

Then, *in your manuscript*, you write the name wrapped in double braces:

EDIT · a paragraph in your book

```
The {{product}} handbook, version {{version}}, begins here.
```

When you export — to the web, but also to PDF or any other format — Inkhaven replaces `{{product}}` with `Tidewalker` and `{{version}}` with `3.0`. Rename the product in the one settings line and every mention in the book updates at once.

INSIGHT

The two kinds are worth keeping straight. `site` values, from `html.hjson`, are for the *website's furniture* — the title bar, the footer — and reach only your templates. `docs.variables`, from `inkhaven.hjson`, are for the *book's own words* and reach every export in every format. One dresses the site; the other edits the prose.

WHAT YOU LEARNED

- **HJSON** is a comfortable `name: value` file format; put site-wide values in `html.hjson`.
- Inkhaven hands that file to templates as `site` — `author: "…"` becomes `{{ site.author }}`.
- Just creating `html.hjson` with `title`/`subtitle`/`author`/`footer` fills the default templates in.
- `docs.variables` in `inkhaven.hjson` is a separate kind: `{{name}}` written **in your prose** is replaced in every export.

PART IV

Publishing Choices

CHAPTER 6

6

Choosing what goes public

A finished Inkhaven project holds far more than the manuscript. It holds your sources, a glossary, a cast of characters, a map of places, the world behind the story, perhaps an invented language — a whole workshop. When you publish, you decide how much of that workshop joins the book on the website. This chapter is the switchboard.

Everything here produces *one* website. Turning a part on does not make a separate site; it adds a page to the same site, listed in the same contents down the side. The book and its apparatus are published together, as one thing.

What is on the site by default

A plain `inkhaven export html` publishes your manuscript and the two things that normally belong *in* a finished book: its **sources** (a bibliography) and its **glossary**. Everything else in the workshop stays private until you ask for it.

> **INSIGHT**
>
> The default is the safe, sensible one: what a reader would expect between the covers of the book — the chapters, the references, the glossary — goes public; your working material — rough notes, character sketches, the world bible — does not. You never accidentally publish your workshop; you *choose* to.

The switchboard

Which parts are published is controlled by a block in your settings file. Each line is a part of the workshop and a plain `true` or `false`:

EDIT · inkhaven.hjson

```
docs: {
  html: {
    include: {
      sources:    true    # the bibliography (on by default)
      glossary:   true    # your defined terms (on by default)
      characters: true    # the cast of characters
      places:     true    # the places / gazetteer
      world:      true    # the world's description
      language:   true    # the invented language
      mythology:  false   # myths and symbols
      notes:      false   # your private notes — usually leave
off
    }
  }
}
```

Set a part to `true` to publish it, `false` to hold it back. To publish a book with no bibliography, set `sources: false`. To add your cast and your gazetteer, set `characters: true` and `places: true`. There is no command-line flag for these — they live in the settings file, because they are a decision about the book, not about one export.

Each part you switch on becomes its own page at the end of the site, and appears in the contents list beside your chapters. Here is what each one turns into on the page:

`sources`
A **bibliography** — your citations, sorted by author, each formatted as a reference with its link.

`glossary`
Your **defined terms**, laid out for reading.

`characters`
The **cast** — a page of your characters and what you have recorded about each.

`places`
A **gazetteer** — your places and their details.

`world`
Your **world**, written up as a **narrative guide** — its sky, its lands, its waters, its peoples, their livelihood, the laws of magic, and its history, in readable prose drawn from your `world.hjson`.

`language`
Your **invented language** — a **dictionary** presented as a table your readers can **sort by any column and filter as they type**, alongside your grammar.

`mythology`
Your **myths and symbols**.

`notes`
Your **notes**. Almost always left `false` — this is your private desk.

> **NOTE**
>
> A part you switch on but never filled in adds nothing — if your Places book is empty, `places: true` produces no page and no clutter. So you can turn on the parts you intend to grow into, and they appear only once they have something to show.

TRY IT

Turn on one part you have actually written — say your glossary or your list of places. Set it to `true` in the settings block above, run `inkhaven export html -o site`, and open the site. A new entry has appeared in the contents, and clicking it shows that part of your workshop, formatted to match the rest of the book. Turn it back to `false` and export again to confirm it vanishes. That is the whole switchboard.

PITFALL

`notes` is off by default for a reason: your notes are where unfinished thoughts, spoilers, and reminders-to-self live. Turn it on only if you genuinely mean to publish that material. When in doubt, leave it off — you can always switch it on later.

Which book

If your project holds several books, choose the one that becomes the site with `--book-name`, as Chapter 2 showed. Each book makes its own site; to publish two, export each into its own folder.

AT THE TERMINAL

```
inkhaven export html -o site --book-name "The Drowned Atlas"
```

Which edition

The finest control is over *which paragraphs* of the manuscript go out — because one book can quietly contain several editions. You mark a paragraph for an edition in the editor with the tagging chord (`Ctrl+B ]`), giving it a tag of the form `profile:edition:full` or `profile:audience:expert`. A paragraph everyone should see gets no such tag.

TERM **A profile**

A label on a paragraph saying which edition it belongs to, written `dimension:value` — like `edition:full`. At export you ask for an edition, and only the matching paragraphs (plus the unlabelled ones) go out.

Then publish one slice by naming the edition:

AT THE TERMINAL

```
inkhaven export html -o site --profile edition=full
```

The rule is designed to be safe: for each edition you ask for, a paragraph is published only if it *matches* that edition or carries *no label for it at all*. Unlabelled writing appears in every edition; asking for no profile publishes everything. So labels only ever *narrow* — they never expose something you did not mark.

PITFALL

Because asking for no profile publishes every paragraph, do not rely on omitting a profile to hide a draft. If a paragraph must never go public, keep it out of the book or below its finished status — profiles choose between editions, they are not a lock on secrets.

WHAT YOU LEARNED

- The site is **one** website; switching a part on adds a page to it, in the same contents.
- By default you publish the manuscript, the **sources**, and the **glossary**; the rest of the workshop is private.
- `docs: { html: { include: { … } } }` in the settings file turns parts on or off — characters, places, world, language, mythology, notes.
- An empty part adds nothing; `--book-name` picks the book; `--profile` publishes one edition and only ever narrows.

PART V

Going Live

CHAPTER 7

7

Putting it on the internet

You have a folder that is a complete website. The last step is to move that folder to a computer that is always on and connected, so anyone with the address can visit. That computer is called a host, and because your site is self-contained, hosting it is about as simple as web hosting ever gets.

What hosting means

> TERM **Web host**
>
> A computer, run by someone else and always online, that holds your website's files and hands them to anyone who asks. You copy your folder to it once; from then on the host serves your pages to visitors. Many hosts cost nothing for a small site like a book.

> TERM **Static hosting**
>
> Hosting for sites that are just files — HTML, CSS, images — with no program running behind them. That is precisely what Inkhaven builds, so any static host will do, and static hosts are the cheapest, fastest, and most durable kind. When a host mentions "static sites," that is you.

Because the site asks nothing of the outside world, there is no database to set up, no software to install on the host, nothing to keep patched. You are moving a folder of plain files. That is the whole job.

The shape of it

However you host, the steps rhyme:

1. **Build the site.** `inkhaven export html -o site` — the `site` folder is what you publish.
2. **Give the folder to the host.** Some hosts have you drag the folder onto a web page; some watch a folder on a file-sharing service; some pull it from a code repository. All of them, in the end, take your folder.
3. **The host gives you an address.** A web address where your book now lives. Share it.

TERM **The front page**

When a visitor arrives at your address with nothing further, the host looks for a file called `index.html` and shows that. Inkhaven always writes one — it is your book's front page — so visitors land in the right place automatically. This is why the file is named as it is; do not rename it.

Checking before you ship

You never have to guess how the site will look online, because it looks the same offline. Open `site/index.html` on your own machine and read it end to end. Click every chapter in the sidebar. Follow the arrows. What you see is exactly what a visitor will see — that is the gift of a self-contained site.

TRY IT

Before publishing, move the whole `site` folder somewhere else on your computer — the desktop, a memory stick — and open `index.html` from there. It should work identically, with every picture and every link intact. If it does (and it will), you have proven the site carries everything it needs, and it will behave the same on any host.

PITFALL

Publish the *contents* of the `site` folder as the root of your web space, so that `index.html` sits at the top. A common slip is to publish the folder *inside* another folder, so visitors must find `.../site/index.html` instead of just your address. If your front page does not appear on its own, check whether `index.html` ended up one level too deep.

Keeping it current

A book is rarely finished all at once. When you revise — fix a line, add a chapter, change the cover subtitle — you simply export again and replace the folder on the

host. There is no separate website to edit and keep in step with the manuscript: the manuscript *is* the website's source, every time. Change the book, re-export, re-upload. The site is never out of date for longer than it takes to run one command.

INSIGHT

This is the quiet promise of the whole feature. Your book and your website are not two things you must maintain in parallel — they are one thing, seen two ways. You keep writing the book; the website is only ever a fresh printing of it, one command away.

WHAT YOU LEARNED

- A **web host** is an always-online computer that serves your files; a **static host** is the right, cheap, durable kind for an Inkhaven site.
- Publishing is moving the `site` folder to the host so `index.html` sits at the top; the host gives you an address.
- What you see opening `index.html` on your own machine is exactly what visitors see — check it there first.
- To update, export again and replace the folder; the manuscript is always the site's single source.

APPENDIX A

A

Tags, subtypes, and status

Three small things you attach to a paragraph shape what the website does with it — and because they are easy to confuse, this appendix lays out each one plainly: what it is, how to set it, and, where it matters for the web, the full list.

They are genuinely *separate*. A paragraph's **status** says how finished it is. Its **structural subtype** says what kind of block it is. Its **tags** are free-form labels. None of them is the others.

INSIGHT

The quick way to keep them straight: **status** is one value on a ladder (it is not a tag); a **subtype** is chosen from a fixed menu and changes how the paragraph looks; **tags** are words you invent and reuse. Different purposes, different keys, different places they show up on the command line.

Status — how finished a paragraph is

Status is a single value drawn from a fixed ladder, lowest to highest:

EDIT · the status ladder

```
none · napkin · first · second · third · final · ready
```

It is *not* a tag — it is its own property of the paragraph. Set it in the editor with `Ctrl+B r`, which cycles a paragraph up the ladder.

TERM `--status` (and `docs review`)

On export, `--status ready` publishes only paragraphs that have reached `ready` or above, so an unfinished draft never ships. `inkhaven docs review` prints the readiness of every chapter and lists what is still below the line — run it before you publish. Status is what both of these read.

Structural subtypes — what kind of block it is

A subtype turns an ordinary paragraph into a *particular kind* of block — a code listing, a warning, a table. Choose one in the tree with `Ctrl+B m`, which reshapes the paragraph and seeds it with the right skeleton. Each renders as proper, styled HTML on the site:

Code listing
A block of code, shown in a highlighted monospace panel.

Admonition — note / tip
A calm, teal-bordered callout for an aside.

Admonition — warning / caution
A coloured callout that draws the eye to a caveat.

Procedure
A numbered list of steps.

Table
A grid of rows and columns.

Math
A formula.

You do not type these as tags — you pick them from the `Ctrl+B m` menu, and Inkhaven records the subtype for you. On the website they become the callout boxes, code panels, and lists you see throughout a well-made page.

Tags — your own labels

Tags are words *you* attach to paragraphs and reuse across the book. Open a paragraph and press `Ctrl+B ]` to open the tag picker:

`A`	Add a new tag — type the word and it joins the project's tag list.
`Space`	Select one or more existing tags from the list.
`T`	Apply the selected tags to the open paragraph.
`D`	Delete a tag everywhere in the project.

`Ctrl+B }` opens the matching search — pick a tag to see every paragraph that carries it.

Two kinds of tag matter when you publish.

Profile tags — editions
A tag of the form `profile:dimension:value` marks a paragraph as belonging to one edition — for example `profile:edition:full` or `profile:audience:expert`. At export, `--profile edition=full` publishes only the matching paragraphs (plus the unlabelled ones). This is the whole of Chapter 6; the tag is how you mark a paragraph in the first place.

Plain tags — slices and organisation

Any other word — `draft`, `appendix`, `chapter-notes` — is a plain tag. Beyond helping you find and organise paragraphs, a plain tag can slice an export: `--tag draft` publishes only the paragraphs carrying `draft`. (Profiles narrow by *edition*; a plain `--tag` narrows to a single labelled set.)

> **PITFALL**
>
> Neither tags nor profiles are a lock on secrets. Anything you leave unlabelled is published in every edition, and a plain export publishes every finished paragraph. To keep material off the site for certain, keep it out of the book or below `ready` — do not rely on the *absence* of a tag.

> **WHAT YOU LEARNED**
>
> - **Status** (`Ctrl+B r`) is a single value on a ladder — not a tag — and drives `--status` and `docs review`.
> - **Structural subtypes** (`Ctrl+B m`) — code, admonitions, procedure, table, math — decide how a paragraph renders.
> - **Tags** (`Ctrl+B ]`: A add · Space select · T apply · D delete) are your own labels.
> - **Profile** tags (`profile:dim:value`) drive `--profile` editions; **plain** tags drive `--tag` slices.

AFTERWORD

About the author

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. His career runs through monitoring and telemetry platforms and into federated observability: systems that must make sense of millions of data points without losing the thread. Observability, in the end, is a discipline of *coherence* — of never reporting a state the system cannot account for. It is not an accident that a tool he built for writers carries the same instinct.

What makes him slightly unusual is a tendency to write his own tools — not small utilities, but programming languages. The Bund language — its compiler, its virtual machine, its document store — lives in a long series of Rust crates on crates.io (`rust_dynamic`, `rust_multistack`, `bundcore`, and more), each a building block that exists because the off-the-shelf options did not fit the shape of the work. Inkhaven is cut from the same cloth.

Why a book just about publishing

Inkhaven can already turn a manuscript into a PDF, an e-book, a printed volume. Adding the web was not about chasing another format for its own sake. It was about

reach: a website is the one form of a book that anyone, on any device, can open without owning, buying, or installing a thing. For an author who cannot afford a publisher — and Inkhaven is built first for those authors — a self-contained website is the shortest honest path between finishing a book and being read.

That is also why this short guide exists. Publishing to the web is surrounded by jargon and by tools that assume you are an engineer. None of that should stand between a writer and their readers. So the feature was built to be simple, and this book was written to explain it in plain words — so that *one command* and *one folder* is genuinely all it takes, and everything past that is yours to shape if you wish.

A work of love

Inkhaven is open source, under a permissive licence: you can read it, fork it, study it, modify it, and pass it on. Strictly, the licence also lets you sell it. The author would, gently but firmly, ask that you do not. Inkhaven was not made as a product. It is a work of love for the authors who can least afford to pay for software — the novelist on a battered laptop, the game-master choosing between rent and tools, the graduate student, the engineer drafting in the same terminal where they write code.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the websites it builds never will either — that is the whole point of *self-contained*. Nothing you make with it reports back to anyone, including its author.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet. If this book helps you put your own book in front of readers, that is enough.

Where to find more

`GitHub` — @vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.

LinkedIn	/in/vladimirulogov — posts on observability, the occasional long-form essay.
YouTube	@vulogov — talks and walkthroughs from the conference trail.
X / Twitter	@vladimir_ulogov

If the export ever gets in the way of publishing instead of out of it, open an issue on GitHub. The author reads them.

END OF THE BOOK · INKHAVEN 3.0.0